



UNIVERSITÀ DI PISA

Artificial Intelligence and Data Engineering
Cloud Computing

Project Documentation

Author:
Zeynaddin Papakhov

Academic Year: 2025–2026

Wikimedia Pageview Analytics with Hadoop MapReduce

Contents

1	Introduction	2
1.1	Application Scenario	2
1.2	Analytics Workflow	2
1.3	The Distributed Processing Workflow	3
2	Dataset description	4
3	Architecture Description	6
3.1	Data Storage and Computation Flow	6
3.2	Project Structure	6
4	Implementation	8
4.1	Hadoop	8
4.1.1	Implementation Choices and Optimizations	8
4.1.2	MapReduce workflow pseudocode	9
4.2	Sequential Baseline	11
4.3	Requirements Coverage	11
5	Experimental evaluation	12
5.1	Default Experimental Baseline Configuration	12
5.2	Comparative Analysis of Execution Times based on Dataset Size	12
5.2.1	Sequential Architecture and Memory Threshold	13
5.3	Comparative Analysis of Execution Times based on Reducers	14
5.3.1	Hadoop Reducer Performance	14
5.4	Comparative Analysis of Memory Usage based on Dataset Size	15
5.4.1	Sequential Memory Behavior	15
5.4.2	Hadoop Memory Behavior	16
5.4.3	Comparative Conclusion	16
6	Conclusion	17

Chapter 1

Introduction

1.1 Application Scenario

Modern web platforms generate large volumes of access events every hour. Each user request, page visit, and automated access can be recorded as a log-like event that describes what was requested and how frequently it was accessed. Wikimedia projects represent a realistic example of this scenario, since their public pageview dumps contain large-scale hourly traffic information across many languages, projects, and pages.

The application scenario of this project is a batch web-traffic analytics pipeline for Wikimedia Pageviews data. The system is designed to ingest raw pageview files, transform them into structured records, and compute aggregate indicators that summarize the behavior of the traffic over a selected time window.

The target users of this type of application are data engineers, system administrators, and analysts who need to understand which projects and pages receive the highest traffic and how traffic changes over time. The workload is suitable for cloud-oriented data processing because it consists of many independent records that can be parsed and aggregated in parallel.

1.2 Analytics Workflow

The application computes four main analytical outputs:

- **Total traffic volume:** the total number of views in the selected time window.
- **Project ranking:** the top Wikimedia projects by number of views.
- **Page ranking:** the top pages by number of views, represented as `project|page`.
- **Hourly traffic trend:** the total number of views grouped by hour.

This workflow combines parsing, filtering, aggregation, ranking, and time-based statistics extraction. It is intentionally more complex than a single counter because the page-level ranking may involve millions of distinct keys. This high-cardinality component is also what makes the workload interesting from a scalability perspective.

1.3 The Distributed Processing Workflow

The distributed workflow is divided into the following stages:

1. **Data ingestion:** hourly Wikimedia Pageviews files are collected and uploaded to HDFS.
2. **Parsing and cleaning:** raw text lines are parsed into structured records. Malformed records and non-positive view counts are discarded.
3. **Intermediate representation:** valid records are serialized into a custom Hadoop Writable and written as SequenceFiles.
4. **Metric generation:** structured records are transformed into analytical keys such as `TOTAL_VIEWS`, `PROJECT:<project>`, `PAGE:<project>|<page>`, and `HOURL:<hour>`.
5. **Shuffle and aggregation:** Hadoop redistributes intermediate key-value pairs to reducers, where partial values are summed.
6. **Result generation:** the final reducer output is written as JSON-like summaries containing totals, rankings, and hourly statistics.

The same analytical objective is implemented in a Python sequential baseline. This baseline is used as a non-parallel reference point for performance comparison.

Chapter 2

Dataset description

The dataset used in this project is the Wikimedia Pageviews dataset. It is publicly available from the Wikimedia dumps service and contains hourly pageview records for Wikimedia projects. Each input file corresponds to one hour of traffic.

A typical input record follows the structure:

```
project page_title view_count response_size
```

Only the first three fields are used by the analytics pipeline. The response size field is present in the raw data but is not required for the selected analytical objectives.

The dataset was selected for three main reasons. First, it is a real web dataset rather than a synthetic benchmark. Second, it naturally supports time-window analysis because it is organized by hour. Third, it contains a high number of distinct projects and pages, making it appropriate for ranking and aggregation workloads.

The experiments use consecutive hourly files from one day. The following configurations were used:

Subset	Files	Compressed	Decompressed Text	HDFS Storage
1h	1	51.7 MiB	185.6 MiB	103.5 MiB
4h	4	199.9 MiB	723.2 MiB	399.8 MiB
8h	8	400.6 MiB	1.42 GiB	801.3 MiB
16h	16	845.4 MiB	2.97 GiB	1.7 GiB

Table 2.1: Dataset sizes used in the experiments. HDFS storage includes replication factor 2.

The original files are stored as gzip-compressed Wikimedia hourly dumps. Hadoop stores these compressed files in HDFS, but during MapReduce execution it reads the decompressed text records. Therefore, the decompressed size better represents the logical amount of text processed by the jobs, while the HDFS storage column represents the physical space used by HDFS replication.

During the ETL phase, every valid input line is transformed into a structured record containing:

- the Wikimedia project identifier;
- the page title;

- the hour extracted from the input filename;
- the number of views.

Chapter 3

Architecture Description

The system is deployed on a single virtual machine configured with Hadoop in pseudo-distributed mode. In this configuration, Hadoop daemons such as HDFS NameNode, HDFS DataNode, YARN ResourceManager, and YARN NodeManager run on the same VM. This setup follows the single-node Hadoop cluster model and allows MapReduce jobs to be executed through the Hadoop ecosystem without requiring multiple physical machines.

3.1 Data Storage and Computation Flow

The raw input files are stored in HDFS for the Hadoop MapReduce implementation. HDFS provides the storage layer used by the MapReduce jobs. The sequential baseline, instead, reads the same files directly from the local filesystem.

The computation flow is structured as follows:

1. input files are prepared locally on the VM;
2. Hadoop input files are uploaded to HDFS under `/user/hadoop/single_project/input/`;
3. the first MapReduce job reads the HDFS input and writes structured SequenceFile output to an intermediate HDFS directory;
4. the second MapReduce job reads the intermediate SequenceFiles and writes the final analytics output to HDFS;
5. benchmark scripts collect execution time and memory-related metrics and save them as CSV files.

This architecture is important when interpreting the results. Since all services run on one VM, the system does not benefit from true multi-node data locality or distributed hardware parallelism. Nevertheless, Hadoop still performs job scheduling, container allocation, HDFS I/O, intermediate writes, and shuffle operations. These overheads explain why Hadoop is slower than the sequential baseline for smaller completed datasets.

3.2 Project Structure

The project is organized into the following main directories:

- `hadoop-pageviews/`: Java Hadoop implementation and Maven configuration;
- `sequential/`: Python sequential baseline;
- `scripts/`: dataset preparation, HDFS upload, benchmark, fetch, and plot scripts;
- `results/benchmark/`: CSV benchmark results and generated plots;
- `docs/`: LaTeX documentation and generated PDF report.

Chapter 4

Implementation

4.1 Hadoop

4.1.1 Implementation Choices and Optimizations

The Hadoop implementation consists of two non-iterative MapReduce jobs in cascade, both written in Java.

- **Job 1: ETL.** The first job parses raw Wikimedia Pageviews lines, filters invalid records, extracts the hour from the input filename, and writes structured records as SequenceFiles.
- **Job 2: Analytics.** The second job reads the structured intermediate records and computes the final aggregate statistics.

Several Hadoop-specific features were implemented:

- **Custom Writable:** a `PageviewRecord` class is used to pass structured records between the ETL and analytics jobs. This avoids fragile string concatenation and makes the intermediate format typed and explicit.
- **setup() method:** the ETL mapper uses `setup()` to extract the hour from the input filename once per map task.
- **cleanup() method:** the analytics mapper uses `cleanup()` to emit locally aggregated metrics only at the end of the map task.
- **In-mapper combining:** the analytics mapper stores partial sums in a local hash map. This reduces the number of emitted key-value pairs and decreases shuffle pressure.
- **Custom partitioning:** a `MetricsPartitioner` routes metrics according to their semantic category, such as project, page, hour, or total.
- **Multiple reducers:** the implementation accepts the number of reducers as a command-line parameter. The experiments evaluate 1, 2, 4, and 8 reducers.

The top-project and top-page outputs are limited to the top 10 entries. This keeps the output readable and prevents the reducer from storing unnecessary result entries.

4.1.2 MapReduce workflow pseudocode

Algorithm 1: PageviewRecord Custom Writable

Class PageviewRecord implements Writable

```
variables:
    project : String
    page    : String
    hour    : String
    views   : long

write(out):
    serialize project, page, hour, views

readFields(in):
    deserialize project, page, hour, views
```

Algorithm 2: ETL Mapper

```
setup(context):
    inputHour = extract hour from current input filename

map(offset, line):
    fields = split line into project, page, views, responseSize
    if fields are malformed:
        skip line
    if views cannot be parsed or views <= 0:
        skip line

    record = PageviewRecord(project, page, inputHour, views) // responseSize is ignored
    emit(NullWritable, record)
```

Algorithm 3: Analytics Mapper with In-Mapper Combining

```
setup(context):
    metricsCache = empty hash map

map(_, record):
    metricsCache["TOTAL_VIEWS"] += record.views
    metricsCache["PROJECT:" + record.project] += record.views
    metricsCache["PAGE:" + record.project + "|" + record.page] += record.views
    metricsCache["HOURL:" + record.hour] += record.views

cleanup(context):
    for each (metricKey, partialSum) in metricsCache:
        emit(metricKey, partialSum)
```

Algorithm 4: Metrics Partitioner

```

getPartition(metricKey, value, numReducers):
    if metricKey contains ":":
        prefix = substring before ":"
    else:
        prefix = metricKey
    return hash(prefix) modulo numReducers

```

Algorithm 5: Analytics Reducer

```

reduce(metricKey, values):
    sum = total of all values

    if metricKey == "TOTAL_VIEWS":
        update global total
    else if metricKey starts with "PROJECT:":
        update project ranking candidates
    else if metricKey starts with "PAGE:":
        update page ranking candidates
    else if metricKey starts with "HOUR:":
        update hourly totals

cleanup(context):
    write JSON summary:
        total views
        top projects
        top pages
        views by hour

```

Algorithm 6: Driver

```

main(args):
    read reducers, input path, intermediate path, output path, topN
    configure memory limits and analytics.top.n
    delete existing intermediate and output paths

    configure Job 1:
        mapper = ETLMapper
        reducers = 0
        output = SequenceFile<PageviewRecord>
    run Job 1

    configure Job 2:
        mapper = AnalyticsMapper
        partitioner = MetricsPartitioner
        reducer = AnalyticsReducer
        reducers = user parameter
        input = SequenceFile
        output = text/JSON-like summary
    run Job 2

```

4.2 Sequential Baseline

The sequential baseline is implemented in Python. It reads the same Wikimedia Pageviews files from the local filesystem and computes the same metrics using Python dictionaries and counters.

This implementation is intentionally non-parallel. It provides a control point for evaluating whether the Hadoop overhead is justified under the available hardware constraints. For small and medium completed datasets, the sequential script is faster because it avoids YARN startup, HDFS reads, intermediate writes, and shuffle. However, it stores page-level counters inside a single process, which becomes a memory limitation at larger input sizes.

4.3 Requirements Coverage

Requirement	Project implementation
Pseudo-distributed Hadoop VM	Hadoop runs in pseudo-distributed mode on a single VM with HDFS and YARN.
Autonomous dataset selection	Wikimedia Pageviews was selected and motivated as a large web analytics dataset.
Large dataset	The 16h stress test processes more than 100 million records and several GB of logical input.
Application scenario	Web traffic monitoring and pageview analytics.
Analytics workflow	Parsing, filtering, aggregation, ranking, and hourly trend extraction.
Two MapReduce jobs in cascade	Job 1 performs ETL; Job 2 performs analytics.
Java implementation	Both Hadoop jobs are implemented in Java.
In-mapper combining	The analytics mapper aggregates metrics locally and emits partial sums in <code>cleanup()</code> .
Custom partitioning	The custom partitioner groups metrics according to their semantic prefix.
<code>setup()</code> and <code>cleanup()</code>	Used in ETL and analytics mapper/reducer life-cycle methods.
Custom Writable	<code>PageviewRecord</code> is used as the intermediate structured record.
More than one reducer	The benchmarks include 1, 2, 4, and 8 reducers.
Python non-parallel comparison	Hadoop is compared with a sequential Python baseline.
Experimental evaluation	Execution time, memory usage, reducer count, and dataset size are evaluated.

Chapter 5

Experimental evaluation

5.1 Default Experimental Baseline Configuration

To keep the comparison controlled, the same analytical pipeline is executed across all configurations. The dataset-size and memory experiments use 4 Hadoop reducers. The reducer benchmark fixes the dataset at 4h and varies only the number of reducers.

The baseline configuration is summarized as follows:

- **Sequential baseline:** single-threaded Python process reading local files.
- **Hadoop baseline:** Java MapReduce pipeline reading from HDFS and executing through YARN.
- **Reducer count for dataset-size benchmark:** fixed at 4 reducers.
- **Top-N threshold:** fixed at 10 for project and page rankings.
- **Correctness check:** Hadoop and sequential outputs were compared on the 1h dataset.

For the correctness check, both implementations produced the same total number of views on the 1h dataset:

Implementation	Total Views
Hadoop MapReduce	15,459,276
Sequential Python	15,459,276

5.2 Comparative Analysis of Execution Times based on Dataset Size

To evaluate scalability, the same analytical pipeline was executed over increasing dataset sizes. Hadoop was executed with 4 reducers, while the sequential implementation processed the same files locally.

Dataset	Hadoop 4 Reducers (s)	Sequential (s)
1h	117	20
4h	295	75
8h	583	154

Table 5.1: Execution time by dataset size.

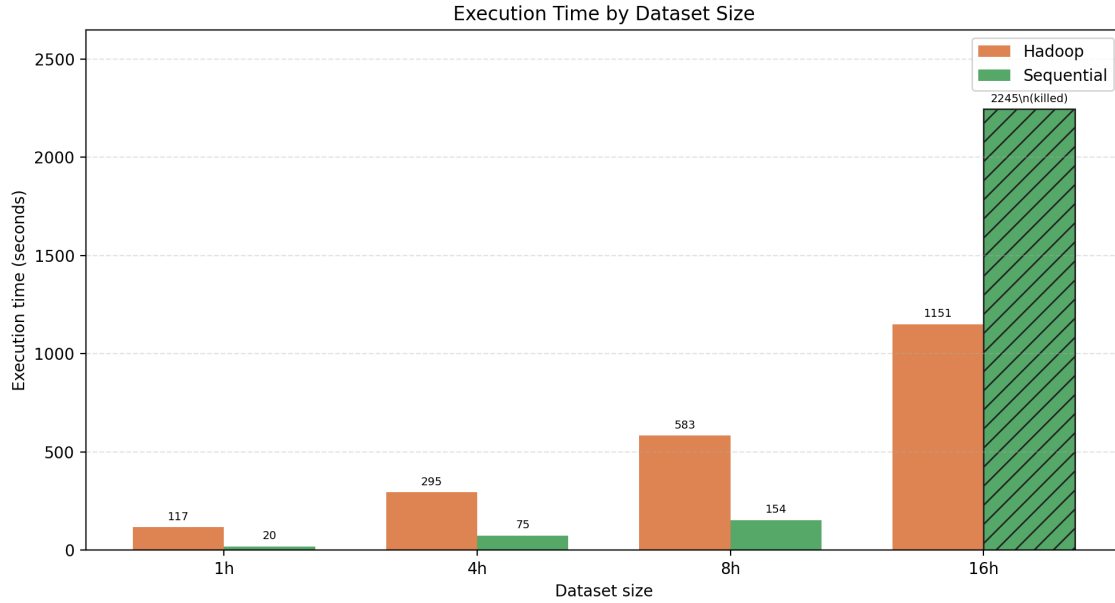


Figure 5.1: Execution time comparison by dataset size.

The results show that the sequential baseline is faster for the completed 1h, 4h, and 8h datasets. This is expected in a single-VM pseudo-distributed environment. Hadoop pays the cost of HDFS access, YARN scheduling, intermediate materialization, and shuffle, while the sequential baseline reads files directly from local storage.

5.2.1 Sequential Architecture and Memory Threshold

The 16h stress test reveals the main limitation of the sequential approach. Hadoop completed the 16h dataset in approximately 1151 seconds, while the sequential Python process was killed after 2245 seconds.

Implementation	Time (s)	Status
Hadoop MapReduce	1151	Completed
Sequential Python	2245	Killed before completion

Table 5.2: 16h stress-test behavior.

The sequential process was killed because the page-level counter is stored inside a single Python process. As the number of unique pages grows, the in-memory dictionary becomes too large for the available VM memory. Hadoop, instead, is able to complete because it relies on disk-backed intermediate data, shuffle, and reducer-based aggregation.

5.3 Comparative Analysis of Execution Times based on Reducers

This experiment evaluates the impact of the number of reducers on Hadoop performance. The dataset is fixed at 4h, while the number of reducers is varied from 1 to 8.

Reducers	Hadoop Time (s)
1	270
2	306
4	308
8	319

Table 5.3: Hadoop execution time by number of reducers on the 4h dataset.

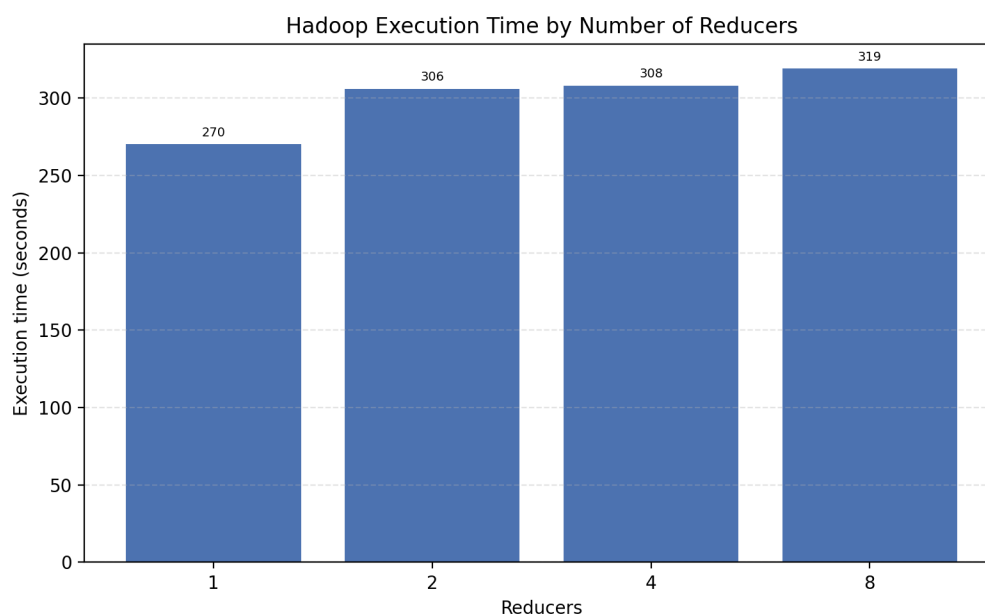


Figure 5.2: Hadoop execution time by number of reducers.

The best runtime is obtained with one reducer. Increasing the number of reducers does not improve performance in this environment. Since the system runs on one VM, additional reducers compete for the same CPU, memory, and disk resources. The extra shuffle and scheduling overhead outweighs the potential benefit of parallel reduction.

5.3.1 Hadoop Reducer Performance

The reducer experiment indicates that the analytics job is strongly affected by framework overhead. With one reducer, the shuffle and output coordination are simpler. With more reducers, Hadoop must manage additional reduce tasks, partitioned shuffle transfers, and multiple output files. On a real multi-node cluster, more reducers could improve parallelism, but on a single VM the benefit is limited.

5.4 Comparative Analysis of Memory Usage based on Dataset Size

The memory benchmark compares Hadoop YARN allocated memory with the peak resident set size of the sequential Python process. These two metrics are not identical: Hadoop memory represents framework-managed allocation, while sequential memory represents the peak memory of a single process. Nevertheless, they show how resource pressure changes with dataset size.

Dataset	Hadoop Avg Allocated MB	Sequential Peak RSS MB
1h	2277.53	991.88
4h	3051.64	2235.91
8h	3334.44	3974.96

Table 5.4: Memory usage by dataset size.

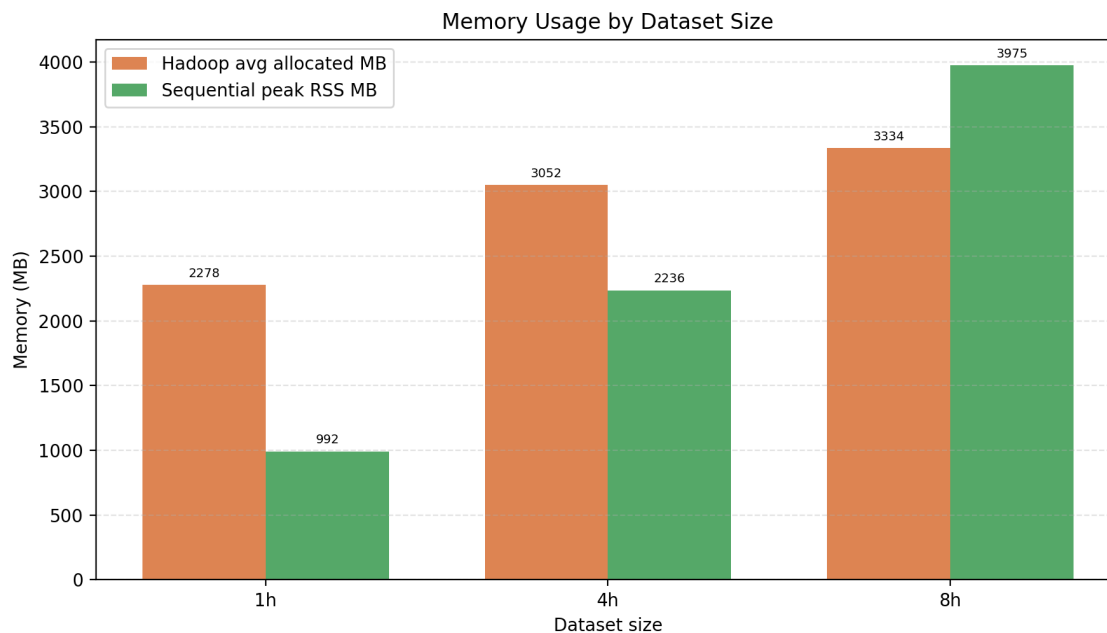


Figure 5.3: Memory usage comparison by dataset size.

5.4.1 Sequential Memory Behavior

The sequential baseline shows a strong increase in memory usage as the dataset grows. This is caused by the page-level dictionary, which must keep one entry for each distinct `project|page` key. At 8h, the sequential process reaches almost 4 GB of peak RSS. At 16h, the same approach no longer completes because the process is killed due to memory pressure.

5.4.2 Hadoop Memory Behavior

Hadoop uses more memory than the sequential baseline at smaller sizes because YARN containers, map tasks, reduce tasks, and the ApplicationMaster require allocated resources. However, Hadoop’s memory behavior is more stable for larger workloads because the framework spills intermediate data to disk and distributes aggregation across reduce tasks.

5.4.3 Comparative Conclusion

The results highlight a trade-off between raw speed and scalability. Sequential processing is faster while the workload fits into memory, but it eventually reaches a memory limit. Hadoop is slower in the single-VM setup, but it can complete larger workloads by relying on disk-backed distributed processing.

Chapter 6

Conclusion

This project implemented a Hadoop MapReduce analytics pipeline for Wikimedia Pageviews data and compared it against a Python sequential baseline. The implementation satisfies the main project requirements: it uses two Java MapReduce jobs in cascade, a custom Writable, setup and cleanup methods, in-mapper combining, a custom partitioner, multiple reducers, and a Python non-parallel baseline.

The experiments show that the sequential baseline is faster for completed 1h, 4h, and 8h datasets because it avoids distributed framework overhead. However, the 16h stress test demonstrates the limitation of this approach: the sequential process is killed due to memory pressure, while Hadoop completes successfully.

The reducer benchmark shows that increasing the number of reducers does not automatically improve performance on a single VM. In this environment, more reducers introduce overhead rather than useful parallelism. Therefore, the results must be interpreted in the context of the pseudo-distributed setup.

Overall, Hadoop MapReduce provides better robustness for larger workloads, while the sequential baseline provides lower overhead for smaller datasets. This is the central trade-off observed in the project.